MODULE 10

ASSIGNMENT: Run the PMF survey honestly — and kill features that don't pass the filter

A joint initiative by

CHECK-IN

What did the market pay you this week?

1. New revenue — how much since last session?
2. Best 'no' — what did the hardest rejection teach you?
3. Repeatable? — has anyone bought the SAME offer twice?

Leaderboard goes up in the workshop — numbers, not vibes

RECAP: Where We Are

1. You ran a real pricing test — at a real price
2. First €500 banked (or the objection log told us why not)
3. Today: what to build next — and what to refuse to build

AI removed the implementation bottleneck. The decision bottleneck remains.

Today

Check-in & leaderboard — what did the market pay you? (15')

IDEA 1 · Refuse most features (15')

IDEA 2 · One North Star, one tree (10')

IDEA 3 · Prioritize like it's money (10')

IDEA 4 · Grow where it compounds (10')

IDEA 5 · Measure PMF honestly (10')

Case · Linear: saying no as strategy (30')

Workshop · Feature Graveyard (90')

Wrap-up · €1,000 pass / €3,000 stretch (10')

From the first €10k to the road to €100k

The ladder is the floor (€1,000 verified at Demo Day). The arc is the ambition: €10k by Demo Day, €100k as the post-course road.

You got here with one feature, 10x better, for a niche. A PRODUCT is that same thing made repeatable — without you in the loop.

Same lean DNA one step further: scale the SYSTEM, not the headcount.

IDEA 1: Refuse most features

AI made building cheap. Deciding stayed expensive.

Every feature passes three questions or dies: asked for? 10x better? would you build it by hand?

The Vibe Coding Death Spiral

AI copilots removed the bottleneck of implementation speed

but NOT the bottleneck of decision quality.

Idea → Excitement → AI builds it fast → Doesn't work perfectly

→ Patch → Patch → Patch → Spaghetti → Project too complex

→ Motivation drops → New project → Repeat

VENTURE LABS

"'I can build it' stopped being a good reason to build it. The question is no longer CAN you build it. It's SHOULD you build it."

-- Vibe Coding Wisdom

The 3-Question Filter

Before starting ANY feature, ask all three

1. Has someone asked for this? → Validates real demand
2. Does this make the product 10x better? → Ensures differentiation
3. Would I build this by hand? → Maintenance honesty check

All 3 Yes → Build it | Any No → Capture the idea, move on

Decision Matrix in Practice

Feature | Asked for? | 10x better? | Build by hand? | Verdict

User-requested export to CSV | ✓ | ✓ | ✓ | BUILD

Generic plugin system for v1 | ✗ | ✗ | ✗ | SKIP

Custom field types (like Notion) | ✗ | ✗ | ✗ | SKIP

One-click deploy for your niche | ✓ | ✓ | ✓ | BUILD

Dark mode | ✓ | ✗ | ✗ | SKIP

If in doubt, don't build. Capture the idea in a backlog and move on.

Anti-Patterns 1 & 2

1. The Generalization Trap

   'If I make the data model generic, it can handle 100x more use cases'

   Reality: More generic = less useful for each specific case

   Rule: Earn generalization through multiple successful specific applications.

2. Infrastructure Before Users

   Building plugin systems, SDKs, credential stores before anyone uses v1

   Reality: Infrastructure for hypothetical users is waste.

   Rule: Build infrastructure only when concrete usage demands it.

Anti-Pattern 3: The Patch Spiral

Feature doesn't work → patch → patch → patch → spaghetti

v1: Doesn't work → Patch #1 → Patch #2 → Spaghetti

Two failed attempts means the design is wrong, not the implementation.

Rule: Doesn't work after 2 tries? Delete everything. Redesign from scratch.

Anti-Patterns 4 & 5

4. Complexity Blindness

Needing extensive docs/memory files to remember your own project's gotchas

Rule: Simplify until a fresh dev can understand it without context docs.

5. The AI Yes-Man Problem

AI builds everything you ask for, never pushes back on scope

Rule: You are the PM. The AI is the contractor.

Key Shifts

AI is the builder → YOU are the builder, AI is the advisor

Ship everything you can build → Ship the smallest thing that tests your hypothesis

Internal excitement = signal → External feedback = signal

Code is cheap to generate → Every line is maintenance debt that feels free

IDEA 2: One North Star, one tree

Pick the single metric that means value delivered, break it into three drivers with owners.

A metric without an action attached is decoration.

Why Product Management Matters

You've validated. You've built. Now you must PRIORITIZE.

100+ feature requests from users | Data-driven prioritization

Limited engineering hours | Clear success metrics

Technical debt accumulating | Systematic roadmap process

Stakeholders want different things | Stakeholder alignment frameworks

'Startups don't starve, they drown.' — Too many priorities = no priority.

The North Star Metric

The ONE metric that captures your core value delivery

Airbnb: Nights Booked - More nights = more value for hosts & guests

Slack: Daily Active Users - More DAU = more team collaboration

Stripe: Payment Volume - More volume = more customer revenue

Your Turn: What single metric represents the core value your product delivers?

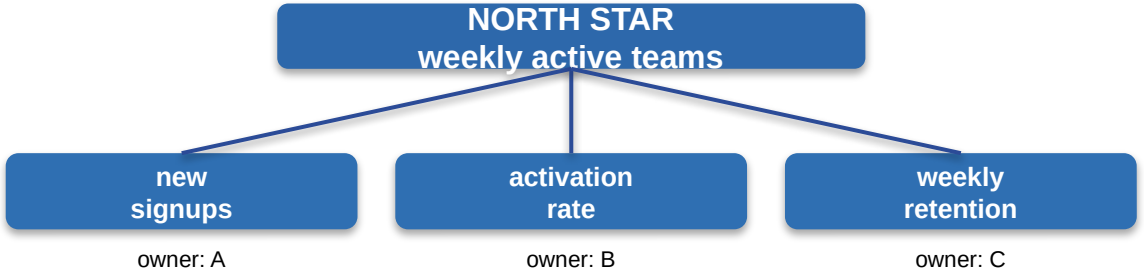
The KPI tree — from North Star to Monday morning

Your North Star says value is delivered. The tree says who does what about it.

North Star = f(drivers). Example: weekly active teams = new signups × activation rate × weekly retention.

Every driver gets an owner and a lever. A metric without an action attached is decoration.

Your dashboard this week: North Star + 3 drivers + revenue. Nothing else.



Data-driven, super lean

Your entire stack: payment dashboard + one spreadsheet + Make.com automations + a 30-minute weekly metrics ritual.

No Snowflake. No data team. If a tool needs an onboarding call, you don't need the tool (yet).

Automate the top 3 recurring ops — onboarding email, invoice, status update. Every automation buys back founder-hours for selling.

Data-driven means you can answer 'what changed last week and why' in one minute. That's it.

AARRR: Pirate Metrics (Dave McClure, 500 Startups)

ACQUISITION

How do users find you? Landing page visits, sign-ups

ACTIVATION

Do they have a great first experience? Completed onboarding, first action

RETENTION

Do they come back? DAU/MAU, return visits

REFERRAL

Do they tell others? NPS, viral coefficient

REVENUE

Do they pay? Conversion rate, ARPU, LTV

IDEA 3: Prioritize like it's money

RICE-score the backlog, keep a Now/Next/Later roadmap.

Pay tech debt when it blocks features — not before, not on principle.

Then hang the roadmap on a heartbeat — an external deadline you don’t set and can’t move.

RICE Scoring: A Deep Dive

Score = (Reach × Impact × Confidence) / Effort

REACH: How many users will this impact per quarter? (Example: 10,000 users)

IMPACT: How much will this move your North Star? (3=Massive, 2=High, 1=Medium, 0.5=Low, 0.25=Minimal)

CONFIDENCE: How certain are you? (100%=High, 80%=Medium, 50%=Low)

EFFORT: How many person-months to build? (Example: 2 person-months)

Example: Auto-save → 5,000 × 2 × 80% / 0.5 = 16,000

Example: Dark Mode → 2,000 × 1 × 100% / 2 = 1,000

→ Auto-save wins by 16x. Build it first.

Technical Debt: The Hidden Tax

'Move fast and break things' has a cost. Know when to pay it down.

WHAT IS IT: Hardcoded values, copy-pasted code, missing tests, outdated dependencies

WHEN TO PAY: It blocks feature development, Causes recurring bugs, Slows onboarding devs, Security vulnerabilities, You've validated PMF

The 20% Rule: Allocate ~20% of engineering capacity to tech debt.

Don't let it accumulate beyond 40% of codebase or velocity drops exponentially.

The Now/Next/Later Roadmap

A flexible roadmap format that balances commitment with adaptability

NOW (This sprint / 2 weeks):

High confidence, Committed delivery, Well-defined scope, Team assigned, Success metrics set

NEXT (1-2 months):

Medium confidence, Roughly scoped, Dependencies identified, May change, Directionally correct

LATER (3+ months out):

Low confidence, Vision-level items, Subject to change, No commitments, Strategic direction

Update weekly. Move items between buckets based on learnings.

OKRs: The Goal-Setting Theater

The default answer to 'how do we stay on track': OKRs — Objectives & Key Results, quarterly, cascaded (Intel → Google → every startup deck).

Two failure modes, both fatal at five people:

1. Superficial goals — sandbagged to be safely hittable, or so 'ambitious' that 70% counts as success. Either way: theater.
2. No consequences — miss the number and the quarter simply rolls over. Nothing breaks. Nobody feels it.

OKRs were invented for 5,000-person alignment problems. You are five people with one problem: revenue.

A goal you set yourself, grade yourself, on a date you can move — that's a diary entry, not a commitment.

"The best external heartbeats aren't set by the company but by outside forces — like the holidays or a big conference."

-- Tony Fadell, Build (2022)

Heartbeats: Deadlines You Don't Control

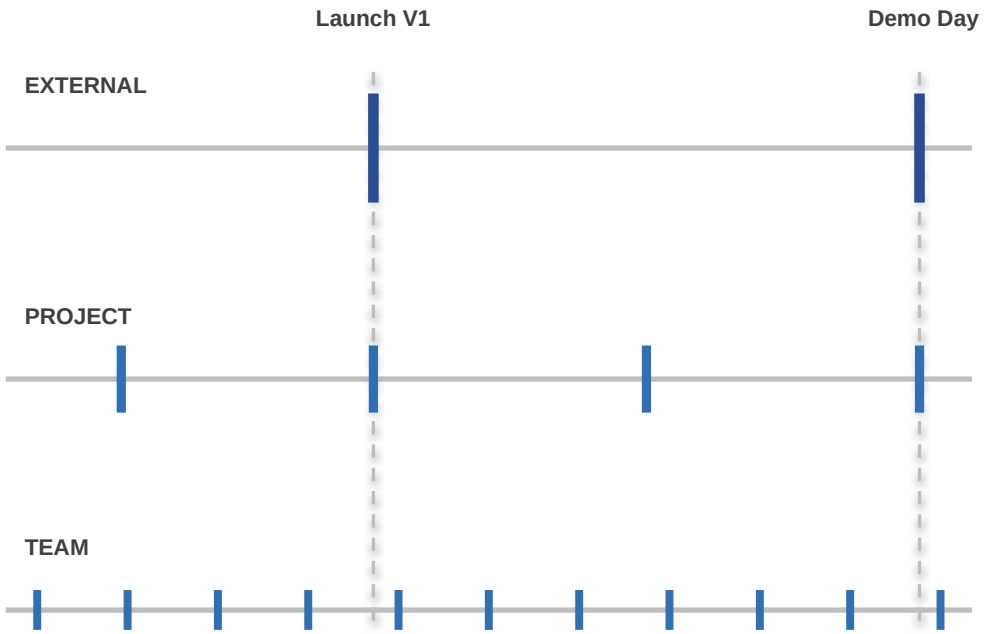
Fadell's fix, from iPod to Nest: run the company on heartbeats — external, project, team.

EXTERNAL · 1–4 events a year that exist whether you like it or not: the trade fair, the buying season, Demo Day. Each beat demands a meaningful improvement — shipped.

That's the difference to OKRs: consequences are built in. Miss the beat and the booth is empty, the press writes about someone else, the customer walks.

PROJECT · milestones back-planned from the external beat. TEAM · the weekly rhythm — you live it every session: leaderboard, numbers, public.

Discipline, not thrash: V1 in 9–18 months, then 2–4 real releases a year. Customers need time to catch up.



Team beats roll up into project milestones; project milestones exist to hit the external beat.

IDEA 4: Grow where it compounds

Retention before activation before monetization before acquisition.

Founder hours are the scarcest capital: 40% sell · 40% build · 20% learn.

Early-Stage Growth Levers

Focus on these three before scaling acquisition:

1. ACTIVATION - Get users to the 'Aha!' moment

   Reduce signup friction, Improve onboarding, Guide to first value, Track activation rate

2. RETENTION - Get users to come back

   Email/push notifications, Habit-forming features, Regular value delivery, Track DAU/MAU

3. MONETIZATION - Convert value to revenue

   Clear pricing page, Reduce payment friction, Upsell/cross-sell, Track conversion rate

Sequence Matters: Fix Retention first → Then Activation → Then Monetization → LAST: Acquisition

Founder time: build vs grow

Your scarcest resource isn't money. It's founder hours.

The trap after first revenue: everyone retreats into the codebase. Nobody sells. Growth stalls within a week.

Budget the week like a portfolio: 40% sell · 40% build · 20% measure & learn — names next to hours.

Standing rule: the sales motion from Modules 8–9 never pauses. Someone owns the pipeline every single week.



one founder-week = 100% · names go next to hours · SELL never drops to zero

Operational excellence at 3–5 people

You are not managing a 20-person dev team. You are protecting laser focus for five people.

One owner per driver metric (your KPI tree). Weekly bets, not backlogs. WIP limit: if everything is important, nothing ships.

A great experience is mostly SPEED: answer customers within 2 hours and you outclass every corporate competitor you have.

Distribute work by strengths (M11 will say it again): who sells, who builds, who runs ops — names, not vibes.

IDEA 5: Measure PMF honestly

The Ellis survey is a compass (±20pp at n=20). Retention curves are the court.

Segment like Superhuman did — then improve what fence-sitters need.

PMF is the V1 gate on Moore’s curve — early adopters first; the mainstream buys a later version.

Product-Market Fit Signals

QUANTITATIVE: 40%+ 'very disappointed' (Sean Ellis heuristic — at n=20 that's ±20pp: a compass, not a verdict)

DAU/MAU > 25% (engagement), Monthly growth > 10% organic, Retention curves flatten, LTV > 3× CAC

QUALITATIVE: Users complain when it's down, Word-of-mouth referrals happening

Users asking for invoices (B2B), 'How did I live without this?', Users finding workarounds for missing features

Running the Sean Ellis survey — this week, on real users

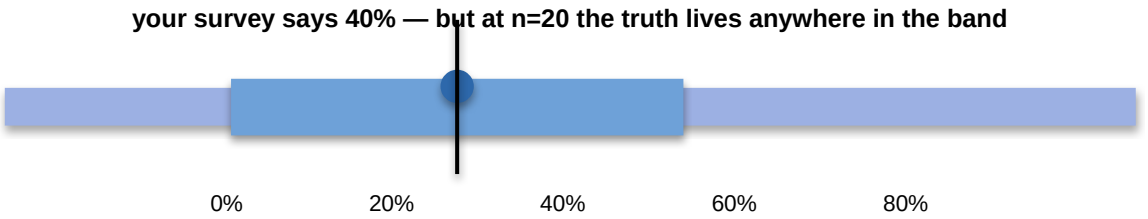
The question: 'How would you feel if you could no longer use [your product]?' — Very / Somewhat / Not disappointed.

Send it to every real user — not friends, not teammates. You'll do it live in the workshop.

Read-out: % 'very disappointed' among ACTIVE users. Then segment by use case — exactly what Superhuman did (Module 9 case).

Honesty at student scale: n=20 means ±20 percentage points. Read the direction, not the decimals.

Pair it with behavior: do people come BACK without being asked? Retention can't flatter you.



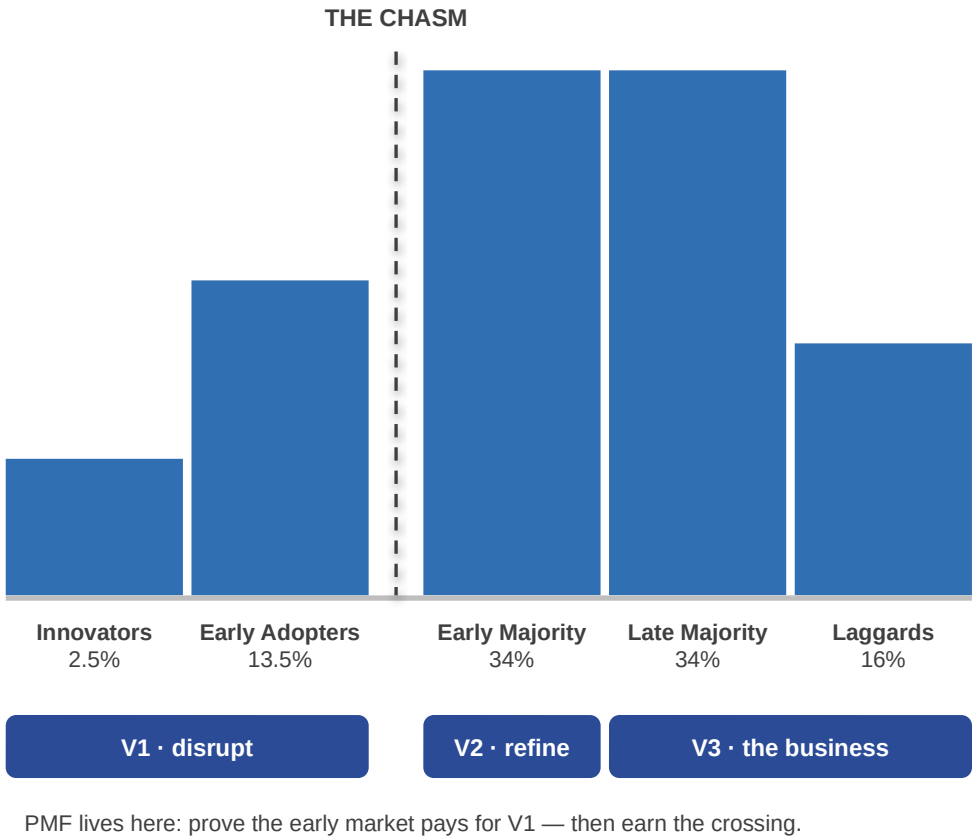
The Chasm: Who Each Version Is For

Moore (1991): the early market and the mainstream buy for different reasons — and between them sits a chasm where products die.

Fadell's overlay: you don't cross it with ONE product. V1 serves innovators & early adopters. V2 the early majority. V3 everyone else.

PMF is the V1 gate: good enough that early adopters BUY. That is the ticket to attempt the crossing — not the crossing itself.

So read your Ellis survey correctly: 40% 'very disappointed' among the tribe V1 is FOR — not among everyone with an email address.



PMF lives here: prove the early market pays for V1 — then earn the crossing.

V1 Disrupts. V2 Refines. V3 Is the Business.

V1 · You're essentially shipping your prototype. Everything is wonky — CAC brutal, features missing. The ONLY question: does a market exist? That's PMF.

V2 · Fix what you screwed up with V1. Widen the market, nail the customer journey → a profitable PRODUCT (gross margin turns positive).

V3 · The product barely changes; the BUSINESS gets reinvented — costs, channels, partnerships, support → net margin.

Nest: V1 (2011) disrupted the thermostat. V2 refined it a year later. V3 looked almost identical — but the company behind it was rebuilt.

You are pre-V1 with twelve weeks on the clock. Earn the right to a V2 — do not polish V3 touchpoints while the V1 question is still open.

This is why IDEA 1 says refuse features: V1 is the minimum feature set for the early market. The 'whole product' is V2's job.

Stakeholder Communication

CO-FOUNDERS

Weekly strategy sync: Shared vision clarity, Agreed priorities, Decision-making process

INVESTORS

Monthly update email: Progress toward milestones, Key metrics trajectory, Major decisions/pivots

TEAM

All-hands + roadmap doc: Clear priorities (why), Context on decisions, Roadmap visibility

The Update Email Template (for Investors)

1. TL;DR (1 sentence: where you are)
2. Key Metrics (3-5 numbers with trend arrows)
3. Wins (2-3 bullet points)
4. Challenges (be honest — ask for help)
5. Key Decisions/Learnings (what you learned)
6. Asks (specific help needed)

What the evidence says

Sean Ellis 40% = practitioner heuristic. Useful, memorable — never peer-reviewed.

The behavioral gold standard: retention curves that flatten (Croll & Yoskovitz, Lean Analytics).

Prioritization is hypothesis testing: write the expected impact BEFORE you build — hypothesis-trained founders outperform (Camuffo et al. 2020, Management Science).

Heartbeats & three generations: Fadell 2022 — the pattern from iPod, iPhone, Nest. A practitioner playbook, not a controlled study.

Crossing the Chasm: Moore 1991 — the segmentation classic; the V1/V2/V3 overlay is Fadell’s, not Moore’s.

Beware metric theater: a KPI you can't act on is a vanity metric with a dashboard.

END RECAP — today in one breath

1 · Three questions or the feature dies

2 · One North Star, three drivers, three owners

3 · RICE + Now/Next/Later — hung on a heartbeat you don’t control

4 · Retention first; founder hours are the scarcest capital

5 · Survey as compass, retention as court — PMF is the V1 gate

€1,000 pass · €3,000 stretch — and the pipeline owner keeps selling.

→ **Next week (Founder Health): the sprint pauses for nobody — but YOU get an honest energy audit before the final push.**

Workshop: Feature Graveyard (90 min)

Agenda lives in M10_Workshop_Feature-Graveyard.md — including the standing 10-minute revenue leaderboard.

Must produce: a culled backlog + your honest PMF score + the improvement plan.

Kill criteria: your backlog still contains a feature nobody asked for.

This week — or it didn't happen

MUST PRODUCE: PMF survey sent + score read honestly + backlog culled to what passes the filter.

PASS BAR: €1,000 cumulative. STRETCH: €3,000.

Homework: M10_Homework.md — run the PMF improvement loop AND keep selling.

Module 11 opens with an honest energy check. Arrive honest.

Thank You!

A joint initiative by

APPENDIX — original slides (16 restored as images)

These pages from the pre-rework deck are not in the current flow. Source snapshot: Instructor/original-decks/M10_Lecture_ORIGINAL.pdf.

Mark each: KEEP (design pass rebuilds it properly, editable) or KILL. Nothing gets deleted without your call.

MODULE 10
Advanced Module
The Intentional Vibe Coder & Product Management

ASSIGNMENT: Apply the 3-Question Filter to your MVP backlog & kill features that don't pass

A joint initiative by

CHECK-IN

Your AI Coding Experience So Far

1. Your Tool - Which AI coding assistant do you use?
2. Biggest Win - What's the best thing you've built with AI?
3. Biggest Fail - Where did an AI project go off the rails?

Share with your neighbor -- 2 minutes each

SECTION A

The Core Problem

A joint initiative by

SECTION C

The 5 Anti-Patterns of Vibe Coding

A joint initiative by

SECTION D

The Healthy Vibe Coding Loop

A joint initiative by

The Healthy Loop

Idea → For whom? → Smallest feature → Who is this for?

→ Validate demand → Minimal scope ↓

← External feedback replaces internal excitement as the signal ←

→ Build yourself → Show 1 user → Iterate or discard

→ AI as reviewer only → Get real feedback

SECTION E

Practical Rules

5 Practical Rules

1. Feature Budget - Maximum 1 new feature per week. Build, deploy, get feedback, iterate.
2. LOC Honesty - AI-generated code counts double as debt. Cheap to write, expensive to understand.
3. Post-Session Review - After every AI session: git diff --stat. Files you didn't request? Ask why, or revert.
4. The Post-It Test - 'This tool does [one thing] for [one person].' Every feature must serve that sentence.
5. Deletion Courage - Deleting code is a feature. Every line removed is maintenance saved.

Discipline beats speed. Every time.

"This tool does [one thing] for [one person]. Every feature must serve this sentence. Put it on a Post-It. Tape it to your monitor."

-- Product Design

DISCUSSION: Questions to Explore

1. When does 'I can build it' stop being a good reason to build it?
2. How does AI copilot availability change the economics of software decisions?
3. How do you maintain architectural ownership when AI writes most of the code?
4. What's the relationship between implementation speed and decision quality?
5. How do you build the discipline to say 'no' when building is effortless?

Break into groups of 3-4. Pick two questions. 10 minutes.

"Build less. Build better. Ship what matters. Now go apply the filter to your own project."

-- Intentional Building

A joint initiative by

Prioritization Frameworks

RICE (Intercom) | ICE (Sean Ellis) | MoSCoW

Reach × Impact × Confidence / Effort | Impact × Confidence × Ease | Must / Should / Could / Won't

Best for: Data-driven teams | Best for: Quick scoring | Best for: Release planning

Have user data? RICE | Running growth experiments? ICE | Aligning stakeholders? MoSCoW

OMTM: One Metric That Matters

At any given time, focus on ONE metric (Lean Analytics, Croll & Yoskovitz)

Problem Validation: # of interviews with 'I need this'

Solution Validation: # of users completing core action

MVP: Activation rate

Growth: Retention (DAU/MAU)

Revenue: MRR or Conversion Rate

Scale: CAC:LTV ratio

Move the OMTM by 10% before changing focus.

WORKSHOP: Roadmap Workshop - RICE Prioritization (60 min)

10 min: Feature Backlog - List ALL requested features and ideas (aim for 20+)

15 min: RICE Scoring - Score each: Reach × Impact × Confidence / Effort

10 min: Stack Rank - Sort by RICE score, identify top 5

15 min: 90-Day Roadmap - Place top features into Now/Next/Later buckets

10 min: Success Metrics - Define how you'll measure success for each feature

Deliverable: Prioritized backlog + 90-day roadmap

REFLECTION: RICE Framework

REACH: How many users will this affect?

IMPACT: How much will it move the needle?

CONFIDENCE: How sure are you about estimates?

EFFORT: How many person-weeks to build?

Score = (Reach × Impact × Confidence) / Effort

Homework: Score your top 10 features with RICE and share roadmap with team

Thank You!

A joint initiative by